

Internal Technical Knowledge and Expert Discovery Platform

Revised Requirements, Architecture and Controlled Implementation Plan

1. Project objective

Build an internal web platform where employees can:

1. Record technical problems they have solved.
2. Document symptoms, error messages, root causes and resolution steps.
3. Search historical problems using natural language.
4. Retrieve relevant previous solutions using hybrid search and RAG.
5. Identify the verified employee who solved the problem.
6. View approved company contact details.
7. Provide feedback on whether the solution worked.

The application must remain focused on:

Technical knowledge storage
Solution discovery
Expert discovery
Employee collaboration

It must not turn into:

A generic chatbot
A full ticketing platform
An employee social network
A project-management system
A Slack or Teams replacement

2. Locked MVP scope

The MVP must include:

- Employee login
- Employee profiles
- Dashboard
- Submit a solved technical problem
- Save drafts
- Edit owned entries
- Root-cause documentation
- Resolution steps
- Code snippets
- Technology tags
- Natural-language search
- Keyword search
- Semantic vector search
- Exact error-message matching
- Bedrock-generated grounded summaries
- Matching solution cards
- Solver details
- Contact action
- Verification status
- Helpful/not-helpful feedback
- Basic reviewer panel
- Responsive interactive UI
- Docker-based local environment
- Minimal AWS deployment
- Automated tests
- Seed data

The MVP will not include:

- Slack integration
- Microsoft Teams integration
- Jira integration
- GitHub synchronization
- Reading private messages
- Reading employee emails
- Automatic employee activity tracking
- Voice input
- Mobile application
- Advanced analytics
- Multi-region deployment
- Enterprise disaster recovery
- Multi-AZ infrastructure
- Amazon RDS

- ECS
 - Kubernetes
-

3. Requirements needed from your side

Required development access

You need:

- GitHub account
- GitHub repository
- AWS account
- AWS Region selected for the project
- Amazon Bedrock model access
- IAM permissions for Bedrock
- Docker Desktop
- Git
- Node.js
- Python
- Code editor
- Internet access for Bedrock API calls

The following commands must work:

```
git --version
docker --version
docker compose version
node --version
npm --version
python --version
```

Bedrock access

The AWS account must have access to:

- One embedding model
- One text-generation model

The application will use Bedrock for:

```
Creating document embeddings
Creating search-query embeddings
Summarizing retrieved solutions
```

Explaining why results matched
Generating grounded answers

Bedrock will not generate:

Employee names
Employee emails
Employee teams
Job titles
Solution ownership
Verification status
Permissions

Those values must come directly from PostgreSQL.

AWS credentials

During development, the backend requires working AWS credentials through one of these supported methods:

AWS CLI profile
Environment variables
EC2 instance role

No AWS keys may be hardcoded into the project.

4. Minimal AWS architecture

The project does not require enterprise-scale AWS infrastructure.

Local development architecture

```
React frontend
  ↓
FastAPI backend
  ├── Amazon Bedrock
  └── PostgreSQL + pgvector in Docker
```

Deployed MVP architecture

```
User browser
  ↓
Nginx
  ↓
React frontend + FastAPI backend
  ↓
PostgreSQL + pgvector
  ↓
Amazon Bedrock
```

For the first deployed version, the application may run using Docker Compose on one AWS EC2 instance.

Containers:

```
nginx
frontend
backend
postgres
```

Persistent PostgreSQL data must be stored on a Docker volume backed by persistent EC2 storage.

AWS services required for the MVP

Only these are required:

AWS service	Purpose
Amazon EC2	Hosts the application containers
Amazon EBS	Stores persistent database data
Amazon Bedrock	Embeddings and grounded answer generation
IAM	Controls Bedrock and deployment permissions
CloudWatch Logs	Optional application logging
Security Groups	Restricts application and SSH access

Explicitly not required initially

```
Amazon RDS
Amazon Aurora
```

Amazon ECS
Amazon EKS
Application Load Balancer
CloudFront
Route 53
Cognito
Multi-AZ databases
NAT Gateway
Multiple deployment environments

These must not be introduced unless a real requirement appears.

5. Database decision

The project requires:

PostgreSQL
pgvector

The project does not initially require:

Amazon RDS

PostgreSQL must run as a Docker container during:

- Development
- Testing
- Initial demonstration
- Initial AWS deployment

Example service:

```
postgres:
  image: pgvector/pgvector:pg16
  environment:
    POSTGRES_DB: knowledge_platform
    POSTGRES_USER: app_user
    POSTGRES_PASSWORD: local_password
  volumes:
    - postgres_data:/var/lib/postgresql/data
```

The application must connect through a standard database URL:

```
DATABASE_URL=postgresql://app_user:password@postgres:5432/knowledge_platform
```

This keeps a future migration to managed PostgreSQL possible without changing the application architecture.

RDS may only be considered when the project has:

- Real company users
- Real sensitive data
- Continuous availability requirements
- Automated backup requirements
- Recovery requirements
- A production budget

Avoiding RDS does not reduce RAG quality.

6. Inputs you should prepare

Employee data

Prepare approximately 10–20 fictional employees.

Each employee should have:

```
Name  
Company email  
Job title  
Team  
Department  
Technical skills  
Profile image or initials  
Application role
```

Technical knowledge entries

Prepare at least 30–50 sample solutions.

Recommended categories:

```
AWS
Terraform
Docker
Kubernetes
Python
React
PostgreSQL
Networking
Linux
Authentication
CI/CD
Security
```

Each entry should include:

```
Title
Problem description
Symptoms
Exact error message
Technology
Environment
Root cause
Resolution steps
Code snippets
Solved by
Date solved
Verification status
Visibility
```

A meaningful retrieval system cannot be tested using only three or four entries.

7. Locked technology stack

Frontend

```
React
TypeScript
Vite
Tailwind CSS
shadcn/ui
Radix UI
Lucide icons
```

Motion for React
React Router
TanStack Query
React Hook Form
Zod

Backend

Python
FastAPI
SQLAlchemy
Alembic
Pydantic
Boto3
JWT authentication

Database

PostgreSQL
pgvector

AI

Amazon Bedrock
Bedrock embedding model
Bedrock text-generation model

The exact Bedrock model IDs must be configured through environment variables so that the architecture does not change if the selected model changes.

Infrastructure

Docker
Docker Compose
Nginx
Amazon EC2
Amazon EBS
GitHub Actions

Codex must not replace these technologies without an approved architecture decision.

8. Repository structure

Use one monorepo.

```
technical-knowledge-platform/
|
├── apps/
|   ├── web/
|   |   ├── src/
|   |   |   ├── components/
|   |   |   ├── features/
|   |   |   ├── pages/
|   |   |   ├── layouts/
|   |   |   ├── hooks/
|   |   |   ├── services/
|   |   |   ├── content/
|   |   |   ├── types/
|   |   |   └── styles/
|   |   └── tests/
|   └── api/
|       ├── app/
|       |   ├── api/
|       |   ├── models/
|       |   ├── schemas/
|       |   ├── repositories/
|       |   ├── services/
|       |   ├── rag/
|       |   ├── security/
|       |   ├── database/
|       |   └── core/
|       └── tests/
|
├── infrastructure/
|   ├── docker/
|   ├── nginx/
|   └── deployment/
|
├── docs/
|   ├── PRD.md
|   ├── ARCHITECTURE.md
|   ├── UI_SPEC.md
|   ├── UI_COPY.md
|   ├── DATABASE_SCHEMA.md
|   └── API_CONTRACT.md
```

```
| | | RAG_DESIGN.md
| | | SECURITY.md
| | | TEST_PLAN.md
| | | IMPLEMENTATION_STATUS.md
| | | CHANGELOG.md
| |
| | .env.example
| | docker-compose.yml
| | README.md
| | CONTRIBUTING.md
```

Codex must not create multiple disconnected backends or duplicate frontend applications.

9. Mandatory documents before implementation

Before writing major application code, create:

PRD.md

Defines:

- Problem
- Users
- Scope
- User stories
- Functional requirements
- Non-functional requirements
- Out-of-scope items
- Acceptance criteria

ARCHITECTURE.md

Defines:

- Frontend architecture
- Backend architecture
- PostgreSQL architecture
- Bedrock integration
- Authentication
- Local deployment
- EC2 deployment
- Security boundaries
- Request flows

DATABASE_SCHEMA.md

Defines:

- Tables
- Columns
- Relationships
- Indexes
- Vector columns
- Visibility rules
- Migration strategy

API_CONTRACT.md

Defines:

- Endpoints
- Request schemas
- Response schemas
- Authentication rules
- Validation rules
- Error response format

RAG_DESIGN.md

Defines:

- Searchable document format
- Embedding generation
- Vector dimensions
- Chunking
- Keyword search
- Exact-error matching
- Hybrid ranking
- Permission filtering
- Similarity threshold
- Reranking
- Prompt structure
- Citation rules
- No-answer behaviour

UI_SPEC.md

Defines:

- Design references

- Screen layouts
- Components
- Spacing
- Typography
- Colours
- Responsive behaviour
- Animations
- Loading states
- Empty states
- Error states
- Accessibility

UI_COPY.md

Defines every static user-facing text string.

This prevents Codex from generating random AI-style copy during implementation.

10. UI design direction

The UI must not be generated from a vague instruction such as:

Create a modern AI dashboard.

That usually produces generic layouts and artificial marketing copy.

Approved design references

The visual direction is:

Linear:
Application shell, sidebar, compact navigation and interactions

Glean:
Search interface, result layout and expert discovery

Notion:
Solution editor, technical documentation and reading experience

We are not copying any product directly.

We are borrowing:

- Information hierarchy
- Interaction patterns
- Layout density
- Search behaviour
- Editor usability
- Navigation patterns

Brand colours, icons, spacing and component styling must remain original.

Template strategy

There will not be one copied dashboard template.

The design process must be:

```
graph TD; A[Reference board] --> B[Wireframes]; B --> C[High-fidelity Figma designs]; C --> D[Clickable prototype]; D --> E[Approved component inventory]; E --> F[React implementation];
```

Reference board
↓
Wireframes
↓
High-fidelity Figma designs
↓
Clickable prototype
↓
Approved component inventory
↓
React implementation

shadcn/ui is only the component foundation. Its default appearance must be customized.

Screens that must be designed before frontend implementation

- Login
- Dashboard
- Global search
- Search results
- Result preview drawer
- Solution details
- Submit solution
- Employee profile
- Reviewer panel
- Empty states
- Error states
- Mobile navigation

11. UI text and interaction requirements

Static website text must be human-written.

Prohibited copy

Do not use text such as:

```
Unlock the power of knowledge
Revolutionize your workflow
Supercharge your productivity
Your AI-powered knowledge companion
Discover limitless possibilities
Welcome to the future of work
```

Approved copy style

Use direct product language:

```
Search past solutions
Log a solved problem
View matching solutions
Contact the solver
Verified by Platform Engineering
Save as draft
Submit for review
No reliable match was found
Add the exact error message to improve your search
```

Only the following may contain AI-generated text:

- Search answer summary
- Explanation of why solutions matched
- Summary of retrieved historical solutions

All navigation labels, headings, buttons, descriptions and empty-state copy must come from `UI_COPY.md`.

Required interactions

- Global command search
- Keyboard shortcut for search
- Expandable search results

- Result preview drawer
 - Search filter chips
 - Hover states
 - Focus states
 - Active states
 - Toast notifications
 - Confirmation dialogs
 - Autosave
 - Draft recovery
 - Copyable code blocks
 - Suggested technology tags
 - Duplicate-problem suggestions
 - Helpful/not-helpful feedback
 - Responsive sidebar
 - Mobile navigation
 - Loading skeletons
 - Network retry controls
-

12. Database model

Core tables:

```
users
employee_profiles
teams
departments
challenges
solutions
solution_contributors
technologies
challenge_technologies
solution_embeddings
attachments
verification_reviews
feedback
search_queries
audit_logs
```

Structured data

These fields must remain structured and must not be generated by Bedrock:

```
Employee name
Email
Job title
Team
Department
Contact information
Solution owner
Verification status
Visibility
Permissions
```

Vector data

The vector table should include:

```
id
solution_id
searchable_text
embedding
embedding_model
content_hash
created_at
updated_at
```

A content hash prevents unnecessary re-embedding when a solution has not changed.

13. Search and RAG design

The search process must follow this order:

1. Authenticate the employee
2. Accept the search query
3. Extract optional filters
4. Create the query embedding through Bedrock
5. Run vector similarity search
6. Run PostgreSQL keyword search
7. Match exact error-message fragments
8. Apply visibility and permission filters
9. Merge results
10. Rerank results
11. Apply a minimum-confidence threshold
12. Retrieve structured employee information

13. Send approved solution context to Bedrock
14. Generate a grounded summary
15. Return sources and solver information

Search methods

Use hybrid retrieval:

Semantic similarity
Keyword matching
Exact error matching
Technology matching
Verification score
Helpful feedback score
Recency

No-answer behaviour

When there is no sufficiently reliable match:

No reliable previous solution was found.

Add the exact error message, affected technology or environment to improve the search.

The system must not invent a probable employee.

Grounding rule

Bedrock may summarize only the solutions retrieved from PostgreSQL.

Every generated answer must reference the original solution records used.

14. Security requirements

The MVP must include:

- Hashed passwords
- JWT authentication
- Role-based access control
- Object-level authorization

- Input validation
- Parameterized database queries
- Restricted CORS
- Upload-size limits
- File-type validation
- Secret scanning
- Rate limiting
- Audit logs
- Safe HTML rendering
- Environment-variable validation
- Bedrock IAM restrictions
- No AWS credentials in source code
- No personal contact details inside vector embeddings

Visibility levels

Company
Department
Team
Restricted
Administrator

Permission filtering must happen before retrieved content is sent to Bedrock.

Initial roles

Employee
Reviewer
Administrator

15. Implementation phases

Phase 0: Requirements and design lock

Deliver:

- PRD
- Architecture
- Database schema
- API contract
- RAG design
- UI reference board

- Wireframes
- High-fidelity UI screens
- UI copy
- Acceptance criteria

Exit condition:

The architecture and UI are approved before feature implementation begins.

Phase 1: Project foundation

Implement:

- Monorepo
- React frontend
- FastAPI backend
- PostgreSQL with pgvector
- Docker Compose
- Alembic migrations
- Health endpoints
- Environment validation
- Seed script
- Basic CI

Exit condition:

```
docker compose up --build
```

starts the complete local system.

Phase 2: Authentication

Implement:

- Login
- Logout
- JWT tokens
- Current-user endpoint
- Protected frontend routes
- Protected API routes
- Employee, reviewer and administrator roles

Phase 3: Knowledge repository

Implement:

- Create challenge
- Add solution
- Save draft
- Edit draft
- Publish solution
- Add technology tags
- Add code snippets
- View solution details
- Employee profiles
- Reviewer verification

Phase 4: UI foundation

Implement approved:

- Design tokens
- App shell
- Sidebar
- Header
- Global search
- Buttons
- Inputs
- Dialogs
- Drawers
- Cards
- Tables
- Badges
- Code blocks
- Loading skeletons
- Empty states
- Error states

No backend feature should invent its own visual pattern.

Phase 5: Keyword search

Implement:

- PostgreSQL text search
- Exact error matching
- Filters
- Sorting
- Pagination

- Search-result page
- Result preview drawer

Search must work without Bedrock before semantic search is introduced.

Phase 6: Bedrock embeddings

Implement:

- Bedrock client
- Embedding service
- Document construction
- Embedding storage
- Query embedding
- Vector similarity
- Re-embedding rules
- Failure handling

Phase 7: Hybrid retrieval

Implement:

- Semantic search
- Keyword search
- Exact-error matching
- Metadata filtering
- Result merging
- Reranking
- Confidence thresholds
- Permission filtering

Phase 8: Grounded Bedrock answers

Implement:

- Context builder
- Grounded prompt
- Source citations
- Structured solver lookup
- No-answer behaviour
- Bedrock timeout handling
- Response validation

Phase 9: Full interactive UI

Implement:

- Search command palette
- Keyboard navigation
- Animations
- Autosave
- Duplicate suggestions
- Responsive design
- Mobile navigation
- Feedback controls
- Toast notifications
- Result expansion
- Contact drawer

Phase 10: Testing and AWS deployment

Implement:

- Unit tests
- Integration tests
- End-to-end tests
- Retrieval evaluation
- Security tests
- Docker production build
- Nginx configuration
- EC2 deployment
- Persistent EBS storage
- Bedrock instance role
- Backup script
- Deployment documentation

16. Definition of done

A feature is complete only when it has:

```
Implementation
Validation
Authorization
Loading state
Empty state
Error state
Success state
```

Responsive state
Automated tests
Updated documentation
No console errors
No broken API requests
No hardcoded production data
Passed acceptance criteria

Rendering one successful page is not enough.

17. Rules Codex must follow

Codex must:

1. Read the project documentation before coding.
2. Implement only the current phase.
3. Preserve the approved stack.
4. Preserve PostgreSQL and pgvector.
5. Preserve Bedrock as the AI provider.
6. Preserve the database schema.
7. Preserve API contracts.
8. Follow approved Figma screens.
9. Use approved UI copy.
10. Add tests for every implemented feature.
11. Fix the smallest possible scope when errors occur.
12. Update implementation status after each task.
13. Explain any remaining blocker clearly.
14. Keep mocks isolated behind interfaces.
15. Keep AWS infrastructure minimal.

Codex must not:

- Add RDS during the MVP.
- Add ECS or Kubernetes.
- Replace PostgreSQL.
- Replace Bedrock.
- replace semantic retrieval with hardcoded results.
- Remove authentication to fix an API.
- Create static fake pages instead of real integrations.
- Invent UI layouts after designs are approved.
- Add generic AI marketing language.
- Change the application into a chatbot.
- Rewrite the repository structure while debugging.
- Mark incomplete features as complete.
- silently introduce another technology.

- remove difficult features because they fail initially.

18. Error-handling protocol

When an error occurs, Codex must:

1. Reproduce the error
2. Record the exact error
3. Identify the failing layer
4. Check the intended architecture
5. Identify the root cause
6. Apply the smallest valid fix
7. Add a regression test
8. Run related tests
9. Confirm existing features still work
10. Continue the same project phase

Codex must not solve a problem by changing the main application architecture.

Example

If Bedrock invocation fails:

Correct response:

```
Check IAM permission
Check Region
Check model ID
Check request format
Handle the Bedrock error
Add a test
```

Incorrect response:

```
Remove Bedrock
Use fake AI responses
Change the project to keyword search only
Mark RAG as complete
```

19. Change-control process

Any major change requires an Architecture Decision Record.

Location:

```
docs/decisions/
```

Example:

```
ADR-001-change-embedding-model.md
```

Each record must contain:

```
Problem
Current architecture
Proposed change
Reason
Alternatives considered
Impact on database
Impact on API
Impact on UI
Migration plan
Rollback plan
Approval status
```

The following changes always require an ADR:

- Database technology
- AI provider
- Authentication approach
- Deployment architecture
- Major UI framework
- API contract
- Repository structure
- Search-ranking strategy

20. Implementation status tracking

Maintain:

docs/IMPLEMENTATION_STATUS.md

Example:

Feature	Status	Tests	UI states	Documentation	Blocker
Project foundation	In progress	Partial	N/A	Yes	None
Authentication	Not started	No	No	Yes	None
Solution submission	Not started	No	No	Yes	None
Keyword search	Not started	No	No	Yes	None
Bedrock embeddings	Not started	No	N/A	Yes	Bedrock access
Hybrid retrieval	Not started	No	No	Yes	Seed data
RAG summaries	Not started	No	No	Yes	None

Allowed statuses:

Not started
In progress
Blocked
Implemented
Tested
Complete

“Complete” may only be used after all acceptance criteria pass.

21. Environment variables

Create a complete `.env.example`.

```
APP_ENV=development
APP_NAME=Technical Knowledge Platform

WEB_URL=http://localhost:5173
API_URL=http://localhost:8000

DATABASE_URL=postgresql://app_user:password@postgres:5432/knowledge_platform

JWT_SECRET=
```

```
JWT_EXPIRE_MINUTES=60

AWS_REGION=
AWS_PROFILE=

BEDROCK_EMBEDDING_MODEL_ID=
BEDROCK_CHAT_MODEL_ID=

UPLOAD_PROVIDER=local
UPLOAD_DIRECTORY=/app/uploads
MAX_UPLOAD_SIZE_MB=10

SEARCH_RESULT_LIMIT=10
SEARCH_SIMILARITY_THRESHOLD=
RAG_ENABLED=true

LOG_LEVEL=INFO
```

The backend must stop during startup with a readable error when required variables are missing.

22. Local run requirements

Startup must remain simple:

```
git clone <repository>
cd technical-knowledge-platform
cp .env.example .env
docker compose up --build
```

Expected URLs:

```
Frontend: http://localhost:5173
Backend: http://localhost:8000
API documentation: http://localhost:8000/docs
```

The seed command must create:

- One administrator
- One reviewer
- Multiple employees
- Teams
- Departments

- Technologies
- Sample challenges
- Sample solutions
- Vector embeddings

The local environment must remain usable even if Bedrock is temporarily unavailable, but semantic search and RAG must clearly display that Bedrock is unavailable rather than returning fake production results.

23. UI acceptance checklist

The UI is acceptable only when:

- It follows approved Figma designs.
 - It has a consistent design system.
 - Navigation is clear.
 - Search is the main interaction.
 - Search results explain why they matched.
 - Employee profiles are well-designed.
 - Forms are broken into meaningful steps.
 - Desktop and mobile layouts work.
 - Loading skeletons are present.
 - Empty states are useful.
 - Error messages are actionable.
 - Hover states exist.
 - Focus states exist.
 - Disabled states exist.
 - Keyboard navigation works.
 - Code blocks are readable.
 - Long errors do not break layouts.
 - Drawers and dialogs work correctly.
 - Animations are subtle and purposeful.
 - No generic gradient-heavy AI design is used.
 - No AI marketing language appears.
 - No raw shadcn default page is accepted.
 - No placeholder text remains.
 - No browser-console warnings remain.
-

24. Testing requirements

Backend unit tests

Test:

- Permissions
- Visibility filtering
- Validation
- Ranking logic
- Embedding-document creation
- Content hashing
- Feedback calculations
- No-answer thresholds

Frontend tests

Test:

- Login form
- Search interactions
- Filter controls
- Result cards
- Result drawer
- Submission steps
- Loading states
- Empty states
- Error states
- Responsive navigation

Integration tests

Test:

- Login
- Challenge creation
- Solution creation
- Verification
- Keyword search
- Vector search
- Hybrid search
- Employee lookup
- Feedback submission

RAG evaluation dataset

Each evaluation record must contain:

```
User query
Expected solution
Expected solver
Expected technology
Expected top-five result
Expected no-answer result
```

Measure:

```
Recall at 5
Correct solver retrieval
Permission-filter accuracy
No-answer accuracy
Citation accuracy
```

End-to-end test

```
Employee logs in
  ↓
Employee records a solution
  ↓
Reviewer verifies it
  ↓
Another employee searches
  ↓
The correct solution appears
  ↓
The solver profile appears
  ↓
The user submits feedback
```

25. Main blockers

Blocker	Effect	Resolution
No Bedrock model access	Semantic search and RAG cannot run	Enable model access
Wrong Bedrock Region	Model invocation fails	Use supported project Region
No AWS permissions	Backend cannot invoke Bedrock	Correct IAM policy
No sample dataset	Search quality cannot be measured	Prepare 30–50 solutions
No approved UI design	Frontend becomes generic	Complete Figma first
No UI copy document	Random AI-style text appears	Complete UI_COPY.md
No permission model	Restricted solutions may leak	Define roles and visibility
No acceptance criteria	Codex marks work complete too early	Use phase exit conditions
No persistent EC2 volume	Database data may be lost	Attach persistent EBS storage
Architecture changes during debugging	Project drifts	Enforce ADR process
Too many AWS services	Cost and complexity increase	Keep minimal architecture
No retrieval tests	RAG sounds good but retrieves incorrectly	Use fixed evaluation dataset

26. Required decisions from your side

Before implementation begins, decide or provide:

1. Project name
2. AWS account
3. AWS Region
4. Bedrock embedding-model access
5. Bedrock generation-model access
6. GitHub repository
7. Brand name
8. Primary accent colour
9. Light mode, dark mode or both
10. Employee roles

11. Visibility levels
12. Approved contact fields
13. Sample employees
14. Sample technical solutions
15. UI reference approval

The following decisions are already locked:

AI provider: Amazon Bedrock
Database: PostgreSQL + pgvector
Initial database hosting: Docker
Initial AWS deployment: EC2 with Docker Compose
Frontend: React + TypeScript
Backend: FastAPI
RDS: Not part of MVP
UI direction: Linear + Glean + Notion references
Static UI copy: Human-written

27. Master instruction for Codex

You are implementing an internal Technical Knowledge and Expert Discovery Platform.

The approved documents inside /docs are the single source of truth.

The locked architecture is:

- React and TypeScript frontend
- FastAPI backend
- PostgreSQL with pgvector
- PostgreSQL running through Docker for the MVP
- Amazon Bedrock for embeddings and grounded answer generation
- Docker Compose for local development
- Docker Compose on one EC2 instance for the initial deployment
- Human-approved UI designs and interface copy

Do not add Amazon RDS, ECS, EKS, Kubernetes, Aurora, Cognito, an Application Load Balancer or other large AWS services unless an approved Architecture Decision Record explicitly requires them.

Do not change the product scope, technology stack, database, AI provider, folder structure, API contracts or design system without an approved Architecture Decision Record.

Before coding:

1. Read PRD.md.
2. Read ARCHITECTURE.md.
3. Read UI_SPEC.md.
4. Read UI_COPY.md.
5. Read DATABASE_SCHEMA.md.
6. Read API_CONTRACT.md.
7. Read RAG_DESIGN.md.
8. Read SECURITY.md.
9. Read TEST_PLAN.md.
10. Read IMPLEMENTATION_STATUS.md.

Implement only the current phase in IMPLEMENTATION_STATUS.md.

For every feature:

1. State the acceptance criteria.
2. Implement the smallest complete vertical slice.
3. Add input validation.
4. Add authorization.
5. Add loading, empty, error and success states.
6. Follow approved Figma designs.
7. Use only approved interface copy.
8. Add automated tests.
9. Run relevant tests.
10. Update documentation.
11. Do not mark the feature complete until all criteria pass.

When an error occurs:

1. Reproduce the exact error.
2. Identify the failing layer.
3. Find the root cause.
4. Apply the smallest correction within the approved architecture.
5. Do not remove the intended feature.
6. Do not switch technologies.
7. Do not insert fake production behaviour.
8. Add a regression test.
9. Confirm existing features still work.
10. Record any remaining blocker.

Amazon Bedrock is a mandatory project dependency.

If Bedrock is temporarily unavailable, display a clear service-unavailable state. Do not replace it with silently generated mock results.

The UI must not look like a generic AI-generated dashboard. Follow the approved Linear, Glean and Notion-inspired design specification. Do not create marketing language, unnecessary gradients, meaningless statistics or random dashboard cards.

Do not proceed to the next phase while the current phase has failing tests, incomplete UI states, undocumented changes or unmet acceptance criteria.

28. Final project principle

The project must be built in this order:

```
Define
Design
Approve
Implement
Test
Review
Complete
```

It must not be built in this order:

```
Generate everything
Encounter errors
Replace difficult features
Add unnecessary AWS services
Change the architecture
Continue with a different application
```

The core project architecture is intentionally simple:

```
React
↓
FastAPI
├─ PostgreSQL + pgvector
└─ Amazon Bedrock
```

The project's quality will come from:

- Strong data modelling
- Accurate hybrid retrieval

- Proper Bedrock integration
- Verified employee information
- Good access control
- Human-designed UI
- Useful interactions
- Reliable testing

It will not come from adding unnecessary cloud infrastructure.